주제: LLM 모델 선정, 지식 베이스 및 RAG 구축

1. RAG의 개념과 필요한 이유

LLM은 학습할 때 본 데이터로만 답하기 때문에, 두 가지 한계가 있다.
- 학습 이후의 최신 정보나 우리 팀만 가진 데이터는 모른다.
- 모르는 내용도 그럴듯하게 지어내서 답한다. ⇒ **환각(hallucination)**

RAG(Retrieval-Augmented Generation, 검색 증강 생성)는 이 문제를 줄이는 방법이다. 질문이 들어오면 먼저 관련 자료를 검색하고, 찾은 자료를 LLM에게 함께 넘겨서 '이 자료를 근거로 답하라'고 시키는 구조이다. LLM이 기억에 의존하지 않고 주어진 자료를 보고 답하기 때문에, 데이터를 바꾸면 LLM을 다시 학습시키지 않아도 답이 달라진다.

- 예시: "강릉에서 조용한 호수 보이는 곳 알려줘"
  - RAG 없이 LLM만 쓰면: 실제로 없는 장소나 틀린 위치를 말할 수 있다.
  - RAG를 쓰면: 우리가 모아둔 장소 데이터에서 후보를 먼저 찾고, 그 소개글만 근거로 설명하므로 실제 있는 장소만 나온다.

- RAG는 크게 세 단계이다.
  1. **준비 단계**: 자료를 모아 검색할 수 있는 형태로 저장 (지식 베이스 구축)
  2. **검색 단계**: 질문과 관련 있는 자료를 찾기
  3. **생성 단계**: 찾은 자료를 근거로 LLM이 답 만들기

2. LLM 모델 선정 기준

LLM은 종류가 많아서 "가장 좋은 모델"을 고르기보다 **우리 서비스에 맞는 모델**을 고르는 것이 중요하다.

- 보통 다음 기준으로 비교한다.
  - 성능: 우리가 시킬 일(조건 추출, 요약, 설명 생성)을 정확히 하는지
  - 한국어 지원: 한국어 질문을 이해하고 자연스럽게 답하는지
  - 비용: 호출할 때마다 돈이 드는지, 무료 한도가 있는지
  - 속도: 응답이 얼마나 빨리 오는지
  - 컨텍스트 길이: 한 번에 읽을 수 있는 글의 양 (검색된 자료를 많이 넣을수록 중요)
  - 도구 호출 지원: 6주차 에이전트에서 함수를 호출하게 하려면 필요
  - 운영 환경: 외부로 데이터를 보내도 되는지, 서버(GPU)가 있는지

- 예시: 프로젝트에 "유료 API는 쓰지 않는다"는 원칙이 있다면, 무료 등급이 있는 API 모델과 직접 돌리는 로컬 모델 두 가지를 후보로 둔다.

3. API 모델과 로컬(오픈소스) 모델 비교

구분	API 모델	로컬 모델
방식	외부 서버에 요청을 보내 답을 받음	내 컴퓨터나 서버에서 직접 실행
장점	설치가 쉽고 성능이 높은 편	호출 한도가 없고 데이터가 밖으로 나가지 않음
단점	호출 한도, 비용, 약관 제한	GPU 등 환경이 필요하고 설정이 번거로움

- 예시: 무료 API는 하루 호출 횟수가 정해져 있어서 시연 중에 한도가 차면 멈출 수 있다. 반대로 로컬 모델은 한도 걱정은 없지만, 노트북에서 돌리면 응답이 느릴 수 있다.

- 모델을 정하는 가장 확실한 방법: 같은 질문 몇 개를 여러 모델에 넣어 결과를 직접 비교하는 것

4. 데이터 수집과 지식 베이스의 개념

지식 베이스는 RAG가 검색하고 답의 근거로 삼는 자료 모음이다. RAG의 답은 이 자료를 넘어설 수 없어서, 지식 베이스의 품질이 곧 답변의 품질이다.

- 지식 베이스를 만드는 흐름은 다음과 같다.
  1. **수집**: 문서, 웹 페이지, API, 이미지 등에서 자료 모으기
  2. **정제**: 중복, 빈 값, 불필요한 내용 제거 (3주차 전처리와 연결)
  3. **구조화**: 검색과 활용에 필요한 형태로 정리
  4. **메타데이터 추가**: 출처, 날짜, 지역 같은 부가 정보 붙이기

- 예시: 우리 프로젝트는 TourAPI에서 받은 정보를 장소 카드로 만든다. 장소 카드 하나에는 장소 ID, 이름, 지역, 위도·경도, 소개글, 사진 URL, 사진 라이선스가 들어간다. 이 장소 카드 전체가 지식 베이스가 된다.

5. 청킹(Chunking): 자료를 검색 단위로 나누기

글이 길면 통째로 저장하지 않고 적당한 크기로 잘라서 저장한다.
이 조각을 청크(chunk)라고 합니다. 이유는 두 가지이다.
- 질문과 관련 없는 내용까지 함께 검색되는 것을 막기 위해
- LLM이 한 번에 읽을 수 있는 양에는 한계가 있기 때문에

- 청크 크기에 따른 차이는 다음과 같다.
  - 너무 작으면: 문맥이 끊겨서 의미를 알기 어렵다.
  - 너무 크면: 필요 없는 내용이 섞여 검색 정확도가 떨어진다.

- 보통 문단이나 몇백 자 단위로 자르고, 앞뒤 조각이 조금씩 겹치게 해서 문맥이 끊기지 않게 한다.

- 예시: 20쪽짜리 여행 가이드 PDF라면 문단별로 잘라서 저장한다. 반면 우리 프로젝트의 소개글은 장소당 몇 문단 정도로 짧아서 장소 카드 하나를 한 단위로 두어도 충분하다. 이처럼 청킹 방식은 자료의 길이와 성격에 맞춰 정한다.

6. 임베딩과 벡터 DB

컴퓨터는 글이나 사진의 '의미'를 직접 비교하지 못한다. 그래서 **임베딩(embedding)**으로 글이나 사진을 숫자 목록(**벡터**)으로 바꾼다. 의미가 비슷한 것끼리는 벡터도 가깝게 만들어지므로, 숫자 사이의 거리를 계산하면 비슷한 정도를 알 수 있다.

- 예시: "잔잔한 호수"와 "고요한 호숫가"는 글자가 다르지만 의미가 비슷하므로 벡터가 가깝다. "시끄러운 시장"은 멀리 떨어진다. 우리 프로젝트에서 쓰는 CLIP은 사진과 문장을 같은 공간에 놓기 때문에 사진과 사진, 사진과 문장도 서로 비교할 수 있다.
- 벡터 DB는 이 벡터를 저장해 두고, 질문 벡터와 가장 가까운 것을 빠르게 찾아주는 저장소이다. 대표적인 도구는 FAISS, Chroma 등이고, 몇천~몇만 개 규모에서는 이 정도로 충분하다.
- 임베딩은 저장할 때 한 번만 계산해 두면 되므로, 큰 모델을 써도 비용 부담이 크지 않다.

7. 검색(Retrieval) 단계

사용자 질문도 같은 임베딩 모델로 벡터로 바꾼 뒤, 벡터 DB에서 가장 가까운 상위 k개를 가져온다. 이때 질문과 자료는 반드시 같은 임베딩 모델로 변환해야 한다. 모델이 다르면 숫자의 기준이 달라서 비교가 안 된다.

- 검색 정확도를 높이는 방법은 다음과 같다.
  - **사전 필터**: 조건에 맞지 않는 자료를 먼저 제외한다. 예: 출발지에서 너무 먼 장소는 직선거리로 먼저 뺀다.
  - **메타데이터 필터**: 지역이나 날짜 같은 정보로 범위를 좁힌다.
  - **점수 집계**: 같은 장소의 사진이 여러 장이면 점수를 하나로 합친다. 예: 최댓값 또는 평균을 쓴다.

8. 생성(Generation) 단계와 프롬프트 구성

검색된 자료를 LLM에게 넘길 때는 프롬프트에 역할, 자료, 질문, 규칙을 함께 넣는다. 이때 "자료에 없는 내용은 말하지 말 것"이라는 규칙이 가장 중요하다.

- 프롬프트 구성 예시:
  - **역할**: 너는 국내 여행지 추천 도우미다.
  - **자료**: [장소 카드 3개의 내용]
  - **질문**: 사용자가 올린 사진과 비슷한 분위기의 장소를 추천해줘.
  - **규칙**: 위 자료에 있는 내용만 근거로 쓰고, 자료에 없으면 "정보 없음"이라고 답해라. 추천 이유에는 장소 이름과 근거 문장을 함께 써라.

- 이렇게 하면 LLM은 자기 기억이 아니라 넘겨받은 자료를 읽고 답을 만들게 된다.

9. 환각 방지와 검증

프롬프트로 규칙을 줘도 LLM이 틀릴 수 있으므로, 코드로 한 번 더 확인한다.

- **존재 검증**: 답에 나온 장소 ID가 실제 DB에 있는지 확인한다.
- **인용 검증**: 답에 인용한 문장이 원래 소개글에 있는 내용인지 확인한다.
- **출처 표시**: 어느 자료를 근거로 했는지 함께 보여준다.

- 예시: LLM이 "A 호수를 추천합니다"라고 답했는데 DB에 A 호수가 없다면 그 답은 버리고 다시 생성하도록 한다. 우리 프로젝트의 목표는 "DB에 없는 장소가 결과에 나오는 경우 0건"이다.

10. RAG 구축 전체 흐름 정리

- 준비 단계 (한 번만 수행)
  1. 데이터 수집 (TourAPI 등)
  2. 정제와 장소 카드 생성
  3. 임베딩 계산
  4. 벡터 DB에 저장

- 사용 단계 (질문이 들어올 때마다 수행)
  1. 사용자 입력 (사진 + 조건 문장)
  2. 조건 추출
  3. 벡터 DB에서 검색
  4. 검색된 자료를 LLM에게 전달
  5. 자료를 근거로 답 생성
  6. 존재와 출처 검증 후 결과 제시